Flutter Clean Architecture

Universal Project Blueprint by Huthaifa Ma'in [LinkedIn]

Folder Structure · Layer Responsibilities · File Templates · Data Flow · DI Checklist

---

📚 **SEE ALSO**
Flutter Clean Architecture — Universal API Integration Guide | Flutter Theme System — Professional Light / Dark Mode Implementation | Custom Flutter Widgets Guide

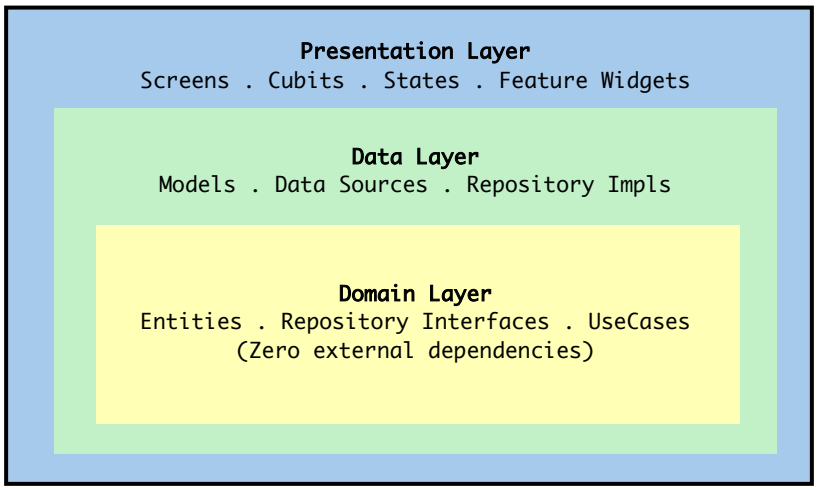
Introduction

This guide is the starting point for every new Flutter project. It defines the universal folder structure, explains the responsibility of every file and folder, and provides a code template for every file that is not covered by a companion guide.

Each file in this document is marked clearly: files that have full templates in a companion guide display a reference banner. Files with no companion coverage include their template and a brief example directly in this document.

The Three Layers

Clean Architecture organises your codebase into three concentric layers. Dependencies only ever point inward — outer layers depend on inner layers, never the reverse.



Layer	Knows About	Must Never Import
Domain	Its own entities, repo interfaces, use cases	Dio, Flutter, Models, Cubits — anything external
Data	Domain entities & interfaces, Dio, JSON, cache	Cubits, screens, Flutter widgets
Presentation	Domain use cases & entities, Flutter, BLoC	Dio, models, raw JSON from data layer directly

> 📌 **NOTE**
> The Dependency Rule: Arrows always point inward. Domain is the innermost layer — zero external package imports, pure Dart only.

Universal Folder Structure

Every project follows this layout. Only endpoint URLs, field names, and Dart class names change between projects — the structure never changes.

```
lib/
├── core/                          # Shared infrastructure
|   ├── api/
|   |   ├── api_consumer.dart       # Abstract HTTP interface
|   |   ├── dio_consumer.dart       # Concrete Dio implementation
|   |   ├── api_interceptors.dart   # Auth headers, error parsing
|   |   ├── api_keys.dart           # JSON key string constants
|   |   ├── connection_checker.dart # Online / offline detection
|   |   └── end_points.dart         # All URL path constants
|   ├── cache/
|   |   └── cache_helper.dart       # SharedPreferences wrapper
|   ├── constants/
|   |   └── mockup_data.dart        # App constants & dev data
|   ├── di/
|   |   └── service_locator.dart    # get_it wiring
|   ├── error/
|   |   ├── error_model.dart        # Server error JSON shape
|   |   ├── error_handler.dart      # DioException → AppException
|   |   ├── exceptions.dart         # AppException hierarchy
|   |   ├── failures.dart           # Failure hierarchy
|   |   └── failures_mapper.dart    # Exception → Failure
|   ├── routing/
|   |   └── app_router.dart         # All routes + guards
|   ├── theming/
|   |   ├── app_colors_extension.dart
|   |   ├── app_styles.dart
|   |   └── theme_cubit.dart
|   └── widgets/                    # Shared UI components
|
├── features/
|   └── [feature_name]/
|       ├── data/
|       |   ├── datasources/[feature]_remote_data_source.dart
|       |   ├── models/[feature]_model.dart
|       |   └── repositories/[feature]_repository_impl.dart
|       ├── domain/
|       |   ├── entities/[feature]_entity.dart
|       |   ├── repositories/[feature]_repository.dart   ← interface
|       |   └── usecases/[action]_usecase.dart
|       └── presentation/
|           ├── manager/[feature]_cubit.dart + [feature]_state.dart
```

```
|           ├── views/[screen]_screen.dart
|           └── widgets/
|
└── main.dart
```

core/api/ — HTTP Client Setup

Everything your app needs to communicate with a backend. The abstract ApiConsumer interface is the critical design decision: all feature code depends on the interface, never on Dio directly. This lets you swap the HTTP client without touching a single feature file.

api_consumer.dart	*Abstract HTTP Interface*

The contract every HTTP implementation must fulfil. Notice it returns Future<dynamic> — the interface is deliberately Dio-free. Feature code and data sources depend on this type, never on DioConsumer directly.

Universal Template — api_consumer.dart

```dart
// No Dio import here - the interface is framework-agnostic.
abstract class ApiConsumer {
  Future<dynamic> get(
    String path, {
    Object? data,
    Map<String, dynamic>? queryParameters,
  });

  Future<dynamic> post(
    String path, {
    Object? data,
    Map<String, dynamic>? queryParameters,
    bool isFormData = false,
  });

  Future<dynamic> patch(
    String path, {
    Object? data,
    Map<String, dynamic>? queryParameters,
    bool isFormData = false,
  });

  Future<dynamic> delete(
    String path, {
    Object? data,
    Map<String, dynamic>? queryParameters,
    bool isFormData = false,
  });
}
```

dio_consumer.dart	*Concrete Dio Implementation*

Implements ApiConsumer using the Dio package. This is the only file that imports package:dio/dio.dart directly. It creates and configures the Dio instance internally, adding the cookie manager, auth interceptor, and log interceptor.

Universal Template — dio_consumer.dart

```
import 'package:cookie_jar/cookie_jar.dart';
import 'package:dio/dio.dart';
import 'package:dio_cookie_manager/dio_cookie_manager.dart';
import '../cache/cache_helper.dart';
import '../error/error_handler.dart';
import 'api_consumer.dart';
import 'api_interceptors.dart';
import 'end_points.dart';

class DioConsumer implements ApiConsumer {
  late final Dio dio;
  final CacheHelper cacheHelper;
  final _cookieJar = CookieJar();

  DioConsumer({required this.cacheHelper}) {
    dio = Dio(BaseOptions(
      baseUrl: EndPoint.baseUrl,
      connectTimeout: const Duration(seconds: 10),
      receiveTimeout: const Duration(seconds: 20),
      sendTimeout: const Duration(seconds: 10),
      headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
      validateStatus: (status) => status != null && status >= 200 && status < 300,
    ));
    dio.interceptors.add(CookieManager(_cookieJar));
    dio.interceptors.add(ApiInterceptor(cacheHelper));
    dio.interceptors.add(LogInterceptor(
      request: true, requestBody: true, responseBody: true, error: true,
    ));
  }

  @override
  Future<dynamic> get(String path, {
    Object? data, Map<String, dynamic>? queryParameters,
  }) async {
    try {
      final r = await dio.get(path, data: data, queryParameters: queryParameters);
      return r.data;
    } on DioException catch (e) { handleDioExceptions(e); rethrow; }
  }

  @override
  Future<dynamic> post(String path, {
    Object? data, Map<String, dynamic>? queryParameters, bool isFormData = false,
  }) async {
```

```
    try {
      final reqData = isFormData && data != null
          ? FormData.fromMap(data as Map<String, dynamic>) : data;
      final r = await dio.post(path, data: reqData, queryParameters: queryParameters);
      return r.data;
    } on DioException catch (e) { handleDioExceptions(e); rethrow; }
  }

  @override
  Future<dynamic> patch(String path, {
    Object? data, Map<String, dynamic>? queryParameters, bool isFormData = false,
  }) async {
    try {
      final reqData = isFormData && data != null
          ? FormData.fromMap(data as Map<String, dynamic>) : data;
      final r = await dio.patch(path, data: reqData, queryParameters: queryParameters);
      return r.data;
    } on DioException catch (e) { handleDioExceptions(e); rethrow; }
  }

  @override
  Future<dynamic> delete(String path, {
    Object? data, Map<String, dynamic>? queryParameters, bool isFormData = false,
  }) async {
    try {
      final reqData = isFormData && data != null
          ? FormData.fromMap(data as Map<String, dynamic>) : data;
      final r = await dio.delete(path, data: reqData, queryParameters: queryParameters);
      return r.data;
    } on DioException catch (e) { handleDioExceptions(e); rethrow; }
  }
}
```

`api_interceptors.dart` — *Request / Response Interceptor*

Runs automatically on every outgoing request and every incoming error. Its jobs are: (1) attach the stored Bearer token to every request header, and (2) clear the stored token on 401 so stale credentials are not retried. Error parsing is handled by error_handler.dart, not here.

Universal Template — api_interceptors.dart

```
import 'package:dio/dio.dart';
import '../cache/cache_helper.dart';
import 'api_keys.dart';

class ApiInterceptor extends Interceptor {
  final CacheHelper cacheHelper;

  ApiInterceptor(this.cacheHelper);

  // ── Attach auth token to every outgoing request ─────────────────
```

```
@override
  Future<void> onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    final token = cacheHelper.getData(key: ApiKeys.token);
    if (token != null) {
      options.headers[ApiKeys.authorization] = 'Bearer $token';
    }
    options.headers['Content-Type'] = 'application/json';
    options.headers['Accept']       = 'application/json';
    handler.next(options);  // always call next — never throw here
  }

  // ── Handle 401: clear stale token ──────────────────────────────
  @override
  Future<void> onError(
    DioException err,
    ErrorInterceptorHandler handler,
  ) async {
    if (err.response?.statusCode == 401) {
      await cacheHelper.removeData(key: ApiKeys.token);
    }
    handler.next(err);  // pass to DioConsumer catch → handleDioExceptions
  }
}
```

📌 **NOTE**
Never throw inside the interceptor's onError — always call handler.next(err). Throwing breaks the error chain. Instead, let the error propagate to the DioConsumer catch block, where handleDioExceptions() converts it into a typed AppException. Never put navigation logic (redirect to login on 401) inside the interceptor. Throw UnauthorizedException there; let the Repository convert it to UnauthorizedFailure; let the Cubit emit the error state; let the screen's BlocConsumer listener navigate.

`api_keys.dart` — *JSON Key String Constants*

Eliminates magic strings from every fromJson method. A typo in a JSON key silently returns null at runtime instead of crashing. Centralising keys here means you catch any mismatch in one place and the compiler helps when you rename a field.

Universal Template — api_keys.dart

```
class ApiKeys {
  // ── Universal keys (appear across multiple feature endpoints) ──
  static const String id        = 'id';
  static const String message   = 'message';
  static const String data      = 'data';
  static const String status    = 'status';
  static const String token     = 'token';
  static const String authorization = 'Authorization';
  static const String errorMessage  = 'error';
```

```
  // ── Auth ──────────────────────────────────────────────────
  static const String email    = 'email';
  static const String password = 'password';
  static const String name     = 'name';

  // ── Feature-specific – add below as needed ────────────────────
  // static const String productPrice = 'price';
  // static const String orderStatus  = 'order_status';
}
```

Feature Example — Using ApiKeys inside fromJson

```
factory ProductModel.fromJson(Map<String, dynamic> json) {
  return ProductModel(
    id:    json[ApiKeys.id],     // 'id'
    name:  json[ApiKeys.name],   // 'name'
    price: json[ApiKeys.price],  // 'price' – add to ApiKeys first
  );
}
```

`connection_checker.dart` — *Network Connectivity Check*

Repository implementations call this before every network request. If the device is offline, they skip the Dio call entirely and immediately return Left(NetworkFailure()). This prevents misleading timeout errors from reaching the UI.

> 📌 **NOTE**
> Requires connectivity_plus package (recommended). Your project may alternatively use data_connection_checker_tv — both packages expose the same interface pattern shown below.

Universal Template — connection_checker.dart

```
import 'package:connectivity_plus/connectivity_plus.dart';

abstract class NetworkInfo {
  Future<bool> get isConnected;
}

class NetworkInfoImpl implements NetworkInfo {
  final Connectivity connectivity;

  NetworkInfoImpl(this.connectivity);

  @override
  Future<bool> get isConnected async {
    final result = await connectivity.checkConnectivity();
    return result != ConnectivityResult.none;
  }
}
```

Usage inside a Repository Implementation

```
@override
Future<Either<Failure, ProductEntity>> getProduct(int id) async {
  if (!await networkInfo.isConnected) {
    return const Left(NetworkFailure('No internet connection.'));
  }
  try {
    final model = await remoteDataSource.getProduct(id);
    return Right(model);
  } on AppException catch (e) {
    return Left(mapExceptionToFailure(e));
  } catch (e) {
    return Left(UnknownFailure(e.toString()));
  }
}
```

`end_points.dart`	*URL Path Constants*

→ Full template and example in the API Integration Guide — Step 1

Universal Template — end_points.dart

```
import 'dart:io';

class EndPoint {
  // Automatically resolves to the correct base URL per platform.
  static String get baseUrl {
    if (Platform.isAndroid) return 'http://10.0.2.2:8000/api/v1';
    if (Platform.isIOS)     return 'http://localhost:8000/api/v1';
    return 'https://your-prod-server.com/api/v1';
  }

  // ── Auth ──────────────────────────────────────────────────
  static const String login    = '/auth/login';
  static const String register = '/auth/register';

  // ── Feature-specific – add below as needed ────────────────
  // static const String products = '/products';
}
```

core/cache/ — Local Storage

`cache_helper.dart`	*SharedPreferences Wrapper*

Every feature that needs to read or write persistent local data uses CacheHelper — never SharedPreferences directly. The auth token, user ID, and saved theme all live here. CacheHelper.init() must be called in main() before runApp().

Universal Template — cache_helper.dart

```dart
import 'package:shared_preferences/shared_preferences.dart';

class CacheHelper {
  static late SharedPreferences _prefs;

  // Call once in main() before runApp()
  static Future<void> init() async {
    _prefs = await SharedPreferences.getInstance();
  }

  Future<bool> saveData({required String key, required dynamic value}) async {
    if (value is String)  return _prefs.setString(key, value);
    if (value is int)     return _prefs.setInt(key, value);
    if (value is bool)    return _prefs.setBool(key, value);
    if (value is double)  return _prefs.setDouble(key, value);
    throw UnsupportedError('Unsupported type: ${value.runtimeType}');
  }

  dynamic getData({required String key}) => _prefs.get(key);

  String? getDataString({required String key}) => _prefs.getString(key);

  Future<bool> removeData({required String key}) => _prefs.remove(key);

  Future<bool> containsKey({required String key}) async => _prefs.containsKey(key);

  Future<bool> clearAll() async => _prefs.clear();
}
```

Feature Example — Save token after login, clear on logout

```dart
// After successful login - save token
await cacheHelper.saveData(key: ApiKeys.token, value: user.token);

// Read token in interceptor
final token = cacheHelper.getData(key: ApiKeys.token);

// On logout - remove all session data
await cacheHelper.removeData(key: ApiKeys.token);
await cacheHelper.removeData(key: AppConstants.userIdKey);
```

⚠️ **CRITICAL**

Never read SharedPreferences directly in widgets or use cases. Always go through CacheHelper. For the theme preference specifically, ThemeCubit's state is the live source of truth — reading the theme key from CacheHelper in widgets gives a stale snapshot.

core/constants/ — App Constants

mockup_data.dart	*App Constants & Dev Mock Data*

Two purposes in one file: AppConstants holds string keys and app-wide values used across the project; MockupData holds hardcoded test values used only during development. Remove MockupData entries before production release.

Universal Template — mockup_data.dart

```
class AppConstants {
  // ── SharedPreferences keys ──────────────────────────────
  static const String tokenKey    = 'auth_token';
  static const String userIdKey   = 'user_id';
  static const String themeKey    = 'theme_mode'; // used by ThemeCubit
  static const String languageKey = 'app_language';

  // ── App-wide values ─────────────────────────────────────
  static const int    paginationLimit = 10;
  static const String appVersion      = '1.0.0';

  // ── Feature-specific – add below ────────────────────────
  // static const String lastSyncKey = 'last_sync_ts';
}

// Development test data – DELETE before production release.
class MockupData {
  static const String mockToken  = 'dev_token_abc123';
  static const int    mockUserId = 1;
}
```

core/di/ — Dependency Injection

service_locator.dart	*get_it Wiring*

→ Full template and feature example in the API Integration Guide — Step 9

> ⚠ **CRITICAL**
> CRITICAL: Cubits must always use registerFactory. All other classes use registerLazySingleton. Registration order is strict: Core → Data Sources → Repositories → Use Cases → Cubits.

Universal Template — service_locator.dart

```
import 'package:get_it/get_it.dart';
import '../api/dio_consumer.dart';
import '../cache/cache_helper.dart';
// ... import your features here

final sl = GetIt.instance;

void setupServiceLocator() {
```

```
  // ── Core ──────────────────────────────────────────────────
  sl.registerLazySingleton<CacheHelper>(() => CacheHelper());
  sl.registerLazySingleton<DioConsumer>(() => DioConsumer(cacheHelper: sl()));

  // ── Data Sources ──────────────────────────────────────────
  sl.registerLazySingleton<AuthRemoteDataSource>(
    () => AuthRemoteDataSourceImpl(dioConsumer: sl()),
  );

  // ── Repositories ──────────────────────────────────────────
  sl.registerLazySingleton<AuthRepository>(
    () => AuthRepositoryImpl(remoteDataSource: sl()),
  );

  // ── Use Cases ─────────────────────────────────────────────
  sl.registerLazySingleton(() => LoginUseCase(repository: sl()));
  sl.registerLazySingleton(() => RegisterUseCase(repository: sl()));

  // ── Cubits - ALWAYS registerFactory ───────────────────────
  sl.registerFactory(() => AuthCubit(loginUseCase: sl(), registerUseCase: sl()));
}
```

core/error/ — Unified Error Handling

Five files work together to ensure every error is caught once, classified, and converted into a domain-safe type. The UI never receives a raw DioException.

```
Dio throws DioException
  |
  ▼  DioConsumer catch block
  handleDioExceptions(e)  ──▶  ServerException / NetworkException / TimeoutException
  |                             (error_handler.dart)
  ▼  repository_impl.dart catch block
  mapExceptionToFailure(e) ──▶  ValidationFailure / UnauthorizedFailure /
  |                             NotFoundFailure / ServerFailure / ...
  ▼  returned as Left(failure)
  result.fold() in cubit ──▶  emit(FeatureError(message: failure.message))
  |
  ▼
  UI shows Snackbar / error widget
```

error_model.dart	*Server Error JSON Shape*

Parses the error response body from the server into a typed object. The fromJson factory handles multiple common backend error shapes automatically. Adjust the field names to match your backend's response.

Universal Template — error_model.dart

```dart
import '../api/api_keys.dart';

class ErrorModel {
  final int? status;
  final String errorMessage;

  ErrorModel({this.status, required this.errorMessage});

  factory ErrorModel.fromJson(Map<String, dynamic> json) {
    // Handle validation error maps: { 'errors': { 'email': ['invalid'] } }
    String extractMessage() {
      if (json['errors'] != null) {
        final errors = json['errors'];
        if (errors is Map) return errors.values.join(', ');
        if (errors is String) return errors;
      }
      return json[ApiKeys.errorMessage]  // 'error'
          ?? json['errorMessage']
          ?? json['message']
          ?? 'Something went wrong';
    }

    return ErrorModel(
      status: json[ApiKeys.status] ?? json['statusCode'],
      errorMessage: extractMessage(),
    );
  }
}
```

Feature Example — fromJson usage

```dart
// Backend returns: { 'error': 'Not found', 'status': 404 }
final errorModel = ErrorModel.fromJson(response.data);
// errorModel.errorMessage == 'Not found'
// errorModel.status == 404

// Backend returns validation errors: { 'errors': { 'email': 'invalid' } }
// errorModel.errorMessage == 'invalid'
```

`error_handler.dart` — *DioException → AppException Converter*

A single top-level function called inside every DioConsumer method's catch block. It maps each DioExceptionType to the appropriate typed AppException, ensuring no raw Dio type ever escapes into the domain layer.

Universal Template — error_handler.dart

```dart
import 'dart:convert';
import 'package:dio/dio.dart';
import 'error_model.dart';
import 'expections.dart';
```

```
void handleDioExceptions(DioException e) {
  final data = e.response?.data;
  Map<String, dynamic>? parsed;

  if (data is Map<String, dynamic>) {
    parsed = data;
  } else if (data is String && data.isNotEmpty) {
    try { parsed = jsonDecode(data); } catch (_) {}
  }

  final errorModel = parsed != null
      ? ErrorModel.fromJson(parsed)
      : ErrorModel(
          status: e.response?.statusCode,
          errorMessage: 'Something went wrong',
        );

  switch (e.type) {
    case DioExceptionType.connectionTimeout:
      throw ServerException(ErrorModel(errorMessage: 'Connection timeout'));
    case DioExceptionType.sendTimeout:
      throw ServerException(ErrorModel(errorMessage: 'Send timeout'));
    case DioExceptionType.receiveTimeout:
      throw TimeoutException(ErrorModel(errorMessage: 'Receive timeout'));
    case DioExceptionType.connectionError:
      throw NetworkException(ErrorModel(errorMessage: 'No internet connection'));
    case DioExceptionType.badResponse:
      throw ServerException(errorModel);   // carries status code for mapper
    case DioExceptionType.cancel:
      throw ServerException(ErrorModel(errorMessage: 'Request cancelled'));
    case DioExceptionType.badCertificate:
      throw ServerException(ErrorModel(errorMessage: 'Invalid SSL certificate'));
    case DioExceptionType.unknown:
      throw ServerException(ErrorModel(errorMessage: 'Unexpected error occurred'));
  }
}
```

exceptions.dart	*AppException Hierarchy*

All exceptions extend AppException. They are thrown only inside data sources (via error_handler.dart) and caught only inside repository implementations. Never throw AppException in use cases or cubits. Each exception carries an ErrorModel so the status code is available to failures_mapper.dart.

Universal Template — exceptions.dart

```
import '../error/error_model.dart';

// All app exceptions extend AppException.
// Thrown in data layer only. Caught in repository implementations only.
abstract class AppException implements Exception {
  final ErrorModel error;
  const AppException(this.error);
```

```
}

class ServerException extends AppException {
  const ServerException(super.error);
}

class NetworkException extends AppException {
  const NetworkException(super.error);
}

class TimeoutException extends AppException {
  const TimeoutException(super.error);
}

class CacheException extends AppException {
  const CacheException(super.error);
}
```

`failures.dart`	*Failure Hierarchy*

The domain-safe equivalent of exceptions. Failures are used in use cases and cubits. This file must never import Dio, http, SharedPreferences, or any data-layer class. Granular subclasses allow the UI to show specific messages per HTTP status code.

Universal Template — failures.dart

```
import 'package:equatable/equatable.dart';

// All failures extend Failure.
// Used in domain layer and presentation (cubits).
// This file has zero data-layer imports.
abstract class Failure extends Equatable {
  final String message;
  const Failure(this.message);

  @override
  List<Object> get props => [message];
}

// HTTP 400 — form/request validation error
class ValidationFailure extends Failure {
  const ValidationFailure(super.message);
}

// HTTP 401 — expired / missing token
class UnauthorizedFailure extends Failure {
  const UnauthorizedFailure([String msg = 'Session expired. Please log in again.'])
      : super(msg);
}
```

```
// HTTP 403 — authenticated but not permitted
class ForbiddenFailure extends Failure {
  const ForbiddenFailure(super.message);
}

// HTTP 404 — resource not found
class NotFoundFailure extends Failure {
  const NotFoundFailure(super.message);
}

// HTTP 409 — data conflict (duplicate, version mismatch)
class ConflictFailure extends Failure {
  const ConflictFailure(super.message);
}

// HTTP 5xx — backend error
class ServerFailure extends Failure {
  const ServerFailure(super.message);
}

// No internet connection
class NetworkFailure extends Failure {
  const NetworkFailure([String msg = 'No internet connection.']) : super(msg);
}

// Request took too long
class TimeoutFailure extends Failure {
  const TimeoutFailure(super.message);
}

// SharedPreferences / local DB error
class CacheFailure extends Failure {
  const CacheFailure(super.message);
}

// Anything that doesn't fit the above
class UnknownFailure extends Failure {
  const UnknownFailure(super.message);
}
```

failures_mapper.dart	*Exception → Failure Converter*

A single top-level function called inside every repository implementation's catch block. It is the bridge between the data layer's exception world and the domain layer's failure world. The status code on the ErrorModel drives granular HTTP failure mapping.

Universal Template — failures_mapper.dart

```dart
import 'exceptions.dart';
import 'failures.dart';

// Converts a data-layer AppException into a domain-layer Failure.
// Called exclusively inside repository implementations' catch blocks.
Failure mapExceptionToFailure(Exception exception) {
  if (exception is TimeoutException) {
    return TimeoutFailure(exception.error.errorMessage);
  }
  if (exception is NetworkException) {
    return NetworkFailure(exception.error.errorMessage);
  }
  if (exception is ServerException) {
    switch (exception.error.status) {
      case 400: return ValidationFailure(exception.error.errorMessage);
      case 401: return UnauthorizedFailure(exception.error.errorMessage);
      case 403: return ForbiddenFailure(exception.error.errorMessage);
      case 404: return NotFoundFailure(exception.error.errorMessage);
      case 409: return ConflictFailure(exception.error.errorMessage);
      default:  return ServerFailure(exception.error.errorMessage);
    }
  }
  return UnknownFailure('Unexpected error');
}
```

Feature Example — How it is used inside a repository implementation

```dart
@override
Future<Either<Failure, UserEntity>> login(String email, String password) async {
  try {
    final model = await remoteDataSource.login(email, password);
    return Right(model);
  } on AppException catch (e) {
    return Left(mapExceptionToFailure(e));   // known error → Failure
  } catch (e) {
    return Left(UnknownFailure(e.toString())); // unexpected crash
  }
}
```

core/routing/ — Navigation

`app_router.dart`	*Routes + Guards*

Maps every named route to its screen widget. Route constants are declared as static strings in the same file so the rest of the app never hard-codes path strings. Route guards (redirects) live here — the most common guard checks for a stored auth token and redirects to login if it is absent.

> 📌 **NOTE**
> Uses the go_router package. Add it to pubspec.yaml: go_router: ^14.0.0

Universal Template — app_router.dart

```
import 'package:go_router/go_router.dart';
import '../cache/cache_helper.dart';
import '../constants/mockup_data.dart';
// import your screens here

abstract class AppRouter {
  // ── Route name constants ──────────────────────────────────────
  // Use these everywhere — never write raw path strings in widgets.
  static const String splash   = '/';
  static const String login    = '/login';
  static const String register = '/register';
  static const String home     = '/home';
  // static const String profile = '/profile';

  // ── Router configuration ──────────────────────────────────────
  static final GoRouter route = GoRouter(
    initialLocation: splash,
    routes: [
      GoRoute(path: splash,   builder: (_, __) => const SplashScreen()),
      GoRoute(path: login,    builder: (_, __) => const LoginScreen()),
      GoRoute(path: register, builder: (_, __) => const RegisterScreen()),
      GoRoute(
        path: home,
        // Route guard — redirect to login if no token stored
        redirect: (_, __) {
          final token = CacheHelper.getData(key: AppConstants.tokenKey);
          return (token == null) ? login : null;
        },
        builder: (_, __) => const HomeScreen(),
      ),
    ],
  );
}
```

Wiring in main.dart — replace MaterialApp with MaterialApp.router

```
MaterialApp.router(
  routerConfig: AppRouter.route,
  theme:      ThemeData(extensions: [AppColorsExtension.light]),
  darkTheme:  ThemeData(extensions: [AppColorsExtension.dark]),
  themeMode:  currentMode,
);
```

Navigating from any widget

```
// Go to a named route (replaces current)
context.go(AppRouter.home);

// Push on top (back button returns)
context.push(AppRouter.profile);

// Replace current route (no back)
```

```
context.pushReplacement(AppRouter.login);

// Pass data between screens
context.go(AppRouter.profile, extra: {'userId': 42});
```

core/theming/ — Theme System

`app_colors_extension.dart`	*Dynamic Color Definitions*

→ Full template, field table, and copyWith / lerp in the Theme System Guide — Step 1

`app_styles.dart`	*Unified Style Interface*

→ Full template, static colors, dynamic getters, text style helpers in the Theme System Guide — Step 2

`theme_cubit.dart`	*Theme State Management*

→ Full file, design decisions, and persistence logic in the Theme System Guide — Step 3

core/widgets/ — Shared UI Components

Any widget used in two or more features belongs here. All shared widgets must consume colors and text styles through AppStyles only — never hard-code colors or font sizes.

> 📚 **SEE ALSO**
> See the Custom Flutter Widgets Guide for complete templates and examples for every shared widget in this folder.

features/ — Feature Slices

Each feature is a self-contained vertical slice. All three layers for a feature live inside one folder. Feature folders never import from other feature folders — only from core/.

`data/datasources/`	*[feature]_remote_data_source.dart*

→ Full templates (POST, GET single, GET list, PUT, DELETE) and feature examples in the API Integration Guide — Step 4

`data/models/`	*[feature]_model.dart*

→ Full templates (flat, nested, sibling, list, merged) and feature examples in the API Integration Guide — Step 3

`data/repositories/`	*[feature]_repository_impl.dart*

→ Full template, the Poly-Trick explanation, and feature examples in the API Integration Guide — Step 6

`domain/entities/`	*[feature]_entity.dart*

→ Full templates (standard, nested, parent-wrapper) and feature examples in the API Integration Guide — Step 2

`domain/repositories/`	*[feature]_repository.dart ← interface*

→ Full template, the Either Rule, and feature examples in the API Integration Guide — Step 5

`domain/usecases/`	*[action]_usecase.dart*

→ Full templates (with parameters, no parameters) and feature examples in the API Integration Guide — Step 7

`presentation/manager/`	*[feature]_cubit.dart + [feature]_state.dart*

→ Full templates (states, cubit, fold rule) and feature examples in the API Integration Guide — Step 8

`presentation/views/`	*[screen]_screen.dart*

→ Full template (BlocProvider + BlocConsumer, listener vs builder rule) and feature examples in the API Integration Guide — Step 10

`presentation/widgets/`	*Feature-specific widgets*

If a widget outgrows this feature and is needed elsewhere, move it to core/widgets/ and check out the Custom Flutter Widgets Guide for templates.

main.dart

→ Full file, startup sequence, and wiring explanation in the Theme System Guide — Step 4

Ensure CacheHelper.init() and setupServiceLocator() are called before runApp(). The correct order is:

Startup sequence

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await CacheHelper.init();          // 1. Local storage ready
  setupServiceLocator();             // 2. All dependencies wired
  final savedTheme = ...;            // 3. Theme read (see Theme guide Step 4)
  runApp(MyApp(initialTheme: savedTheme));
}
```

Data Flow — End to End

```
User Action
  |
  ▼
  Screen ──▶ context.read<FeatureCubit>().performAction(input)
  |   emit(FeatureLoading())
  ▼
  await performActionUseCase(input)               [Domain Layer]
  |
  ▼
  UseCase ──▶ repository.performAction(input)     [interface call]
  |
  ▼  [Data Layer]
```

```
RepositoryImpl ──▶ networkInfo.isConnected?
  YES |
      ▼
      try { remoteDataSource.performAction(input) }
      |
      DioConsumer.post(EndPoint.path, data: {...})
      |
      ApiInterceptor attaches token ──▶ Dio sends request
      |
      on DioException ──▶ handleDioExceptions() ──▶ throws AppException
      |
      JSON response ──▶ Model.fromJson(response['data'])
      |
      return Right(model)
  catch AppException ──▶ Left(mapExceptionToFailure(e))
|
result travels back up through UseCase to Cubit
|
result.fold(
  (failure) => emit(FeatureError(message: failure.message)),
  (data)    => emit(FeatureSuccess(data: data)),
)
|
▼
BlocConsumer rebuilds UI  (spinner → content / error Snackbar)
```

New Feature Checklist

Core Setup (once per project)

- core/api/ — api_consumer.dart, dio_consumer.dart, api_interceptors.dart, api_keys.dart, connection_checker.dart, end_points.dart
- core/error/ — error_model.dart, error_handler.dart, exceptions.dart, failures.dart, failures_mapper.dart
- core/cache/cache_helper.dart — CacheHelper.init() called in main()
- core/constants/mockup_data.dart — AppConstants keys declared
- core/di/service_locator.dart — core infrastructure registered
- core/routing/app_router.dart — initial routes and any auth guards configured
- core/theming/ — AppColorsExtension, AppStyles, ThemeCubit (see Theme System guide)

Domain Layer (write first — no external dependencies)

- Create [feature]_entity.dart
- Create [feature]_repository.dart — abstract, Either<Failure, Entity> return types
- Create one [action]_usecase.dart per action

Data Layer

- Create [feature]_model.dart — extends Entity, fromJson(), fromJsonList() if needed
- Create [feature]_remote_data_source.dart — abstract + Dio implementation
- Create [feature]_repository_impl.dart — single try-catch, Right/Left wrapping

Presentation Layer

- Create [feature]_state.dart — Initial, Loading, Success (entity), Error (message)
- Create [feature]_cubit.dart — use cases injected, result.fold() per method
- Create [screen]_screen.dart — BlocProvider + BlocConsumer
- Create feature-specific widgets in presentation/widgets/ if needed

Wiring

- Add endpoint path to core/api/end_points.dart
- Add new JSON keys to core/api/api_keys.dart if needed
- Register data source, repository, use cases, cubit in service_locator.dart
- Add route to core/routing/app_router.dart

Quick Reference — Which Layer Does This Belong In?

If you need to…	It belongs in
Define what a 'User' is (the concept)	domain/entities/user_entity.dart
Convert User JSON to a Dart object	data/models/user_model.dart
Declare that you can log in	domain/repositories/auth_repository.dart (interface)
Actually make the login Dio call	data/datasources/auth_remote_data_source.dart
Coordinate the login flow	domain/usecases/login_usecase.dart
Catch exceptions and return a Failure	data/repositories/auth_repository_impl.dart
Convert DioException to typed AppException	core/error/error_handler.dart
Parse the server's error JSON body	core/error/error_model.dart
Define exception types thrown in data layer	core/error/exceptions.dart
Define failure types used in domain layer	core/error/failures.dart
Convert exception → failure in one place	core/error/failures_mapper.dart
Attach Bearer token to every request	core/api/api_interceptors.dart
Swap HTTP client without touching features	core/api/api_consumer.dart (abstract)
Store all URL paths	core/api/end_points.dart
Store all JSON key strings	core/api/api_keys.dart
Check online/offline before a network call	core/api/connection_checker.dart
Save/read/delete local persistent data	core/cache/cache_helper.dart
Store SharedPreferences key strings	core/constants/mockup_data.dart (AppConstants)
Show a loading spinner or error Snackbar	presentation/views/[screen]_screen.dart
Manage loading/success/error state	presentation/manager/[feature]_cubit.dart
Make a reusable widget (2+ features)	core/widgets/ → see Custom Widgets guide
Handle light/dark theme + toggle	core/theming/ → see Theme System guide
Register all class dependencies	core/di/service_locator.dart
Define named routes + auth guards	core/routing/app_router.dart

Golden Rules

- Domain never imports from data or presentation. Entities, repository interfaces, and use cases are pure Dart.
- One UseCase per action. GetProducts and CreateProduct are two separate classes.
- The only try-catch in the API flow lives in the repository implementation. Never catch exceptions in data sources or cubits.
- handleDioExceptions() is the only place that maps DioException to AppException. It is called in the DioConsumer catch block only.
- mapExceptionToFailure() is the only place that maps AppException to Failure. It uses the status code from ErrorModel for granular HTTP failure types.
- AppException carries an ErrorModel — not just a string — so the status code is available for precise failure mapping.
- Never throw inside ApiInterceptor.onError — always call handler.next(err) to let DioConsumer handle it.
- Register Cubits with registerFactory. All other classes use registerLazySingleton.
- fromJson keys must match the API doc exactly. A wrong key silently produces null.
- fold() every Either. Never call .getOrElse without handling both sides.
- Listener is for side effects; builder is for drawing widgets. Never navigate inside the builder.
- Widgets import app_styles.dart only — never app_colors_extension.dart directly.
- Name colors by role, never by value — foreground not darkNavyBlue.
- One Entity and one Model per JSON map shape — see the API guide for the merge pattern.
- Never read SharedPreferences directly in widgets. Use CacheHelper; use ThemeCubit state for the active theme.
- CacheHelper.init() and setupServiceLocator() must both be called before runApp().

For complete feature code templates, refer to the API Integration Guide (Steps 1–10). For theming, refer to the Theme System Guide (Steps 1–4). For shared widgets, refer to the Custom Flutter Widgets Guide.